

Sistema de Gestión de Biblioteca con Programación Orientada a Objetos

Memoria del Proyecto · Trabajo Final de Programación

Universidad Europea de Valencia · Grado en Ingeniería Informática

Primer Curso · Curso 2025-2026

Vladyslav Taratunskyi · Alejandro Castro · Álvaro Rovira · Enrique Olmo

Índice de Contenidos

1. Introducción y descripción del proyecto
2. Objetivos
3. Arquitectura del sistema
4. Diseño orientado a objetos: jerarquía de clases
5. Los cuatro pilares de la POO aplicados
6. Decisiones técnicas clave
7. Sistema de usuarios y control de acceso
8. Sistema de préstamos y sanciones
9. Gestión de estados de materiales
10. Configuración centralizada
11. Ventajas del diseño
12. Dificultades encontradas y soluciones
13. Conclusiones

1. Introducción y descripción del proyecto

El presente documento constituye la memoria técnica del proyecto final de la asignatura de Programación del primer curso del Grado en Ingeniería Informática de la Universidad Europea de Valencia. El proyecto ha sido desarrollado por el grupo formado por Vladyslav Taratunskyi, Alejandro Castro, Álvaro Rovira y Enrique Olmo durante el curso 2025-2026.

El proyecto consiste en el diseño e implementación de un sistema integral de gestión para una biblioteca, denominado **Reto Biblioteca POO**. El sistema permite administrar de forma centralizada y automatizada el catálogo de materiales, los usuarios y el ciclo completo de préstamos.

1.1 Problema que resuelve

Una biblioteca real necesita gestionar simultáneamente materiales de naturaleza muy distinta (libros físicos, dispositivos electrónicos, juegos de mesa y recursos digitales con licencias), así como diferentes tipos de usuarios con permisos distintos (socios que reciben préstamos y empleados con distintos niveles de acceso administrativo). Coordinar todo esto de forma manual es propenso a errores, inconsistencias y olvidos. Nuestro sistema digitaliza y automatiza todos estos procesos.

1.2 Tecnología utilizada

El proyecto está desarrollado íntegramente en **Python**, utilizando el paradigma de Programación Orientada a Objetos. La lógica de negocio se encuentra en el módulo **models.py**, completamente desacoplado de la interfaz y de la capa de persistencia. Se hace uso de los módulos estándar **abc** para las clases abstractas, **enum** para los enumeradores y **datetime** con **timedelta** para la gestión de fechas.

2. Objetivos

Los objetivos del proyecto fueron establecidos desde el inicio con el fin de guiar las decisiones de diseño y garantizar un resultado de calidad:

1

Sistema modular y mantenible

Aplicar los principios de la POO para crear un código organizado, reutilizable y fácil de mantener a lo largo del tiempo.

2

Separación de responsabilidades

Aislar completamente la lógica de negocio de la interfaz y de la persistencia de datos, de forma que cada capa pueda evolucionar de forma independiente.

3

Jerarquía de clases con herencia

Modelar los distintos tipos de materiales y usuarios mediante herencia, evitando la duplicación de código y facilitando la especialización.

4

Automatización de préstamos y sanciones

Implementar un sistema inteligente que calcule fechas de devolución automáticamente y aplique sanciones sin intervención humana.

5

Diseño extensible

Crear una arquitectura que permita incorporar nuevos tipos de materiales o usuarios en el futuro sin necesidad de modificar el código existente.

3. Arquitectura del sistema

El sistema se ha diseñado siguiendo una arquitectura de **tres capas independientes**, aplicando el principio de separación de responsabilidades:

Capa	Componente	Responsabilidad	Ventaja
Lógica de Negocio	models.py	Reglas del dominio bibliotecario, clases, validaciones y comportamiento.	Testeable e independiente.
Persistencia	Base de datos	Almacenamiento y recuperación de datos, abstrayendo los detalles del motor.	Intercambiable sin tocar la lógica.
Interfaz	UI / CLI	Interacción con el usuario, presentación de información y captura de acciones.	Adaptable a cualquier tecnología.

Principio clave: si mañana se cambia la base de datos o se sustituye la interfaz por una aplicación web, la lógica de negocio en models.py no necesita modificarse en absoluto.

4. Diseño orientado a objetos: jerarquía de clases

El diseño del sistema gira en torno a dos jerarquías de herencia principales, coordinadas por la clase **Prestamo**, que actúa como punto de unión entre ambas mediante una relación de asociación:

4.1 Jerarquía de materiales

La clase abstracta **Material** define el contrato común para todos los materiales de la biblioteca. De ella heredan dos ramas:

- **MaterialFísico**: representa cualquier material tangible. De esta clase heredan a su vez tres clases concretas: **Libro**, **Dispositivo** y **JuegoDeMesa**, cada una con sus atributos específicos.
- **RecursoDigital**: representa recursos con licencias digitales concurrentes. Su comportamiento es radicalmente distinto al de los materiales físicos, lo que justifica su separación en una rama propia.

4.2 Jerarquía de usuarios

La clase abstracta **Usuario** define la base común para todos los tipos de usuario. De ella heredan:

- **Socio**: cliente de la biblioteca que puede realizar préstamos, con límite de préstamos simultáneos y posibilidad de ser sancionado.
- **Empleado**: personal del sistema con rol asignado y métodos de comprobación de permisos. Nunca realiza préstamos.

4.3 La clase Prestamo

La clase **Prestamo** no hereda de Material ni de Usuario, sino que las conecta mediante asociación. Representa el contrato entre un socio y un material, y gestiona automáticamente el ciclo completo: creación, renovación, devolución y sanción.

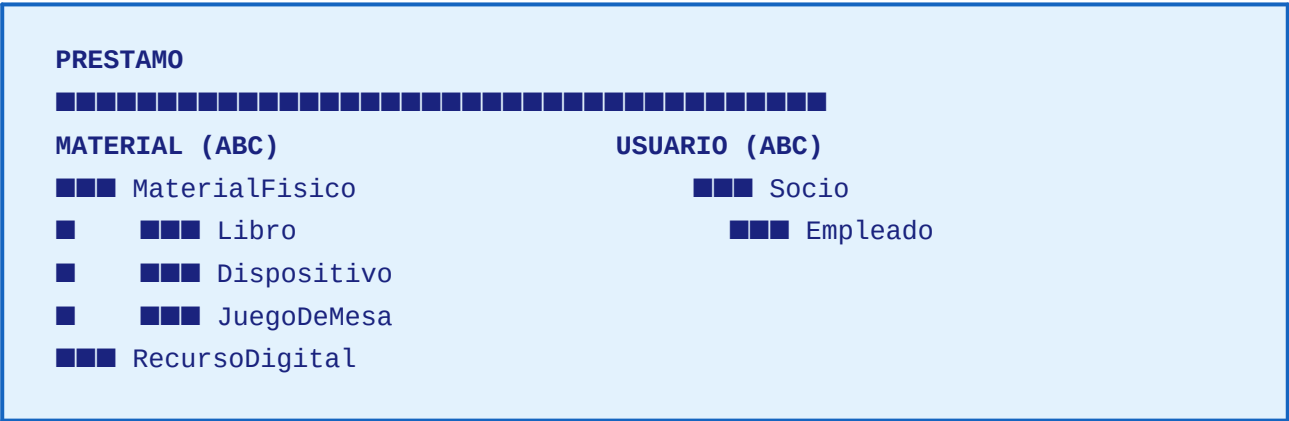


Figura 1. Jerarquía de clases del sistema.

5. Los cuatro pilares de la POO aplicados

Encapsulamiento	Todos los atributos de las clases utilizan la convención de nombres privados (prefijo <code>_</code>) y se exponen únicamente a través de propiedades con getters y setters que validan los datos antes de asignarlos. Esto garantiza que el estado interno de cada objeto solo puede modificarse de forma controlada y validada.
Herencia	La jerarquía de clases permite que Libro, Dispositivo y JuegoDeMesa compartan el comportamiento común de MaterialFisico sin duplicar código. Del mismo modo, Socio y Empleado comparten la base de Usuario. Esto facilita la extensión del sistema con nuevos tipos.
Polimorfismo	Gracias al contrato definido por las clases abstractas, el sistema puede operar sobre cualquier material (sea un libro, un dispositivo o un recurso digital) de forma uniforme, sin necesidad de conocer el tipo concreto. Los métodos <code>prestar()</code> , <code>devolver()</code> y <code>puede_devolverse()</code> funcionan igual para todos.
Abstracción	Las clases Material y Usuario son clases abstractas que definen la interfaz obligatoria sin proporcionar implementación. Esto obliga a cualquier subclase a implementar los métodos esenciales, previniendo errores de diseño y documentando la intención del sistema.

6. Decisiones técnicas clave

6.1 Clases abstractas (ABC)

Se ha utilizado el módulo `abc` de Python con el decorador `@abstractmethod` para definir los métodos obligatorios en Material y Usuario. Los métodos abstractos definidos son: `prestar()`, `devolver()`, `puede_devolverse()` y `descripcion_corta()`.

La razón de esta decisión es garantizar que cualquier nuevo tipo de material o usuario que se añada en el futuro respete la misma interfaz. Si una subclase no implementa alguno de estos métodos, Python lanza un error en el momento de la instanciación, no en tiempo de ejecución cuando ya podría causar fallos en producción.

6.2 Enumeradores

En lugar de utilizar cadenas de texto libres para representar estados y roles, se han definido cuatro enumeradores con la clase `Enum` de Python:

Enumerador	Valores posibles
------------	------------------

EstadoMaterial	Disponible, Pendiente de Recogida, Prestado, No Disponible, Bloqueado
EstadoPrestamo	Activo, Devuelto, Retrasado
TipoDispositivo	Ordenador, Tablet, E-Reader, Calculadora, Otros
RolEmpleado	Bibliotecario, Auxiliar, Administrador

Esta decisión elimina los errores de escritura, facilita el autocompletado en el entorno de desarrollo y centraliza todos los valores posibles en un único lugar del código.

6.3 Properties con validación

Todos los atributos sensibles de las clases se exponen mediante el decorador **@property** con setters que validan los datos antes de asignarlos. La validación se realiza en el modelo, no en la interfaz, por tres razones fundamentales:

- Los datos pueden llegar de múltiples fuentes: formulario de usuario, base de datos, importación masiva o llamada directa desde código.
- Cambiar la interfaz no compromete la integridad del modelo: la validación siempre se aplica.
- El modelo actúa como fuente de verdad: hay un único sitio donde se definen las reglas.

Ejemplos de validaciones implementadas: el email debe contener arroba y dominio, el número mínimo de jugadores no puede superar al máximo, el número de páginas debe ser positivo, y los campos obligatorios no pueden estar vacíos.

7. Sistema de usuarios y control de acceso

Una de las decisiones de diseño más importantes fue separar los tipos de usuario en clases independientes en lugar de usar una sola clase con un atributo de rol.

7.1 La clase Socio

Representa a los clientes de la biblioteca. Sus características principales son:

- Puede realizar préstamos de materiales, con un límite máximo de préstamos simultáneos definido en la configuración.
- Dispone de un control de sanciones: si devuelve materiales con retraso, queda bloqueado para nuevos préstamos.
- Tiene un sistema de permisos especiales para investigadores y usuarios VIP, con un cupo de préstamos ampliado.

7.2 La clase Empleado

Representa al personal de la biblioteca. Sus características principales son:

- Gestiona el catálogo de materiales y los usuarios del sistema.
- Tiene un rol asignado del enumerador RolEmpleado, con tres niveles jerárquicos.
- Nunca realiza préstamos como socio: sus responsabilidades son exclusivamente administrativas.

Nivel	Rol	Permisos
1	Auxiliar	Operaciones básicas del día a día.
2	Bibliotecario	Gestión avanzada del catálogo y bloqueo de materiales.
3	Administrador	Operaciones críticas e irreversibles. Acceso total al sistema.

La separación de Socio y Empleado en clases distintas responde al Principio de Segregación de Interfaces (ISP): cada clase solo tiene los métodos que realmente usa, evitando la acumulación de métodos inútiles que nunca se invocan.

8. Sistema de préstamos y sanciones

La clase **Prestamo** representa el contrato entre un socio y un material, y gestiona automáticamente todo el ciclo desde la reserva hasta la devolución.

8.1 Cálculo automático de fechas

Al crear un préstamo, el sistema calcula automáticamente la fecha de devolución en función del tipo de material:

Tipo de material	Plazo	Justificación
------------------	-------	---------------

Libro	15 días	Período estándar de lectura.
Dispositivo electrónico	7 días	Uso técnico temporal.
Juego de mesa	2 días	Alta rotación entre socios.
Recurso digital	Variable	Según configuración de la licencia.

Los cálculos de fechas se realizan con el módulo **datetime** y **timedelta** de Python, lo que garantiza el correcto manejo de meses de diferente duración y años bisiestos.

8.2 Renovaciones

El sistema permite renovar un préstamo, desplazando la fecha de vencimiento, pero únicamente si el préstamo se encuentra en estado Activo y no ha superado la fecha límite. No tiene sentido renovar un préstamo que ya está retrasado.

8.3 Sanciones automáticas

Cuando un socio devuelve un material, el sistema compara automáticamente la fecha de devolución real con la fecha límite acordada. Si existe retraso, aplica la sanción de forma inmediata y sin intervención humana. El socio sancionado queda bloqueado para nuevos préstamos hasta que un empleado levante la sanción manualmente.

Las ventajas de este enfoque son: eliminación de errores humanos y olvidos, consistencia total en la aplicación de sanciones, liberación del personal de tareas repetitivas, y transparencia para el socio.

9. Gestión de estados de materiales

Uno de los aspectos más complejos del sistema es la gestión de estados, que difiere notablemente entre materiales físicos y recursos digitales.

9.1 Materiales físicos: estados secuenciales

Un material físico solo puede estar en un estado a la vez y sigue una secuencia lineal:

Disponible → Pendiente de Recogida → Prestado → Disponible

9.2 Recursos digitales: pool de licencias

Un recurso digital puede estar prestado a varios usuarios simultáneamente hasta agotar sus licencias. Su estado se calcula de forma matemática comparando las licencias disponibles con las totales. Adicionalmente, un bibliotecario puede bloquear el recurso de forma manual, y esta decisión tiene prioridad absoluta sobre cualquier recálculo automático.

Principio de diseño clave: el método `actualizar_estado()` respeta el bloqueo manual y nunca lo sobrescribe automáticamente. Cualquier sistema serio debe permitir que las decisiones humanas tengan prioridad sobre la automatización cuando es necesario.

10. Configuración centralizada

Se ha creado la clase **ConfiguracionBiblioteca** para centralizar todas las constantes globales del sistema. Las constantes incluidas son:

- Días de préstamo por tipo de material (libros, dispositivos, juegos de mesa).
- Número máximo de préstamos simultáneos por socio.
- Cuota ampliada de préstamos para investigadores y usuarios VIP.
- Valores de sanciones por retraso.
- Límites de renovaciones permitidas.

Esta decisión responde al principio **DRY (Don't Repeat Yourself)** y al concepto de **Single Source of Truth**: si mañana la biblioteca cambia su política de préstamos, basta con modificar un único valor en un único lugar, y todo el sistema se actualiza de forma coherente. No existe riesgo de olvidar modificar el valor en algún otro sitio.

11. Ventajas del diseño elegido

- **Mantenibilidad**

Los cambios están localizados gracias al bajo acoplamiento entre capas y clases. Modificar el plazo de préstamo de libros solo requiere cambiar un valor en ConfiguracionBiblioteca.

- **Escalabilidad**

Añadir un nuevo tipo de material (por ejemplo, un audiolibro) consiste simplemente en crear una nueva subclase que herede de Material y respetar el contrato abstracto. No hay que modificar el resto del sistema.

- **Testabilidad**

La lógica de negocio está completamente aislada de la interfaz y la base de datos, por lo que puede probarse de forma unitaria e independiente sin necesitar ninguna capa externa.

- **Extensibilidad**

Las nuevas subclases heredan automáticamente el comportamiento común y el polimorfismo garantiza que sean tratadas igual que las existentes.

- **Robustez**

Las validaciones integradas en los setters, los estados controlados por enumeradores y las sanciones automáticas hacen que sea muy difícil introducir datos incorrectos o estados inconsistentes.

- **Flexibilidad**

Se puede cambiar el motor de base de datos o la tecnología de la interfaz sin tocar en absoluto la lógica de negocio en models.py.

12. Dificultades encontradas y soluciones adoptadas

Problema 1: Modelado de materiales físicos y digitales

Descripción: Ambos tipos comparten la interfaz de préstamo pero tienen lógicas de estado muy distintas. Mezclarlos en una única clase habría requerido múltiples condicionales que degradarían la legibilidad y el mantenimiento.

Solución adoptada: Se diseñaron dos ramas separadas de herencia (MaterialFisico y RecursoDigital) que comparten el contrato abstracto de Material pero implementan su propia lógica interna de forma limpia e independiente.

Problema 2: Gestión de estados automáticos y manuales

Descripción: El estado de los recursos digitales depende tanto de un cálculo automático (licencias disponibles) como de una posible intervención manual (bloqueo por el bibliotecario). Ambas fuentes podían entrar en conflicto.

Solución adoptada: Se implementó el método `actualizar_estado()` con una comprobación explícita del bloqueo manual. Si el recurso está bloqueado manualmente, el método respeta esa decisión y no la sobreescribe con el recálculo automático.

Problema 3: Evitar la validación duplicada

Descripción: Sin una estrategia clara, la misma lógica de validación tendería a duplicarse en la interfaz y en el modelo, generando inconsistencias cuando una de las dos copias se actualizara sin modificar la otra.

Solución adoptada: Toda la validación se centralizó en los setters del modelo mediante propiedades. La interfaz delega la validación al modelo sin reimplementarla.

Problema 4: Constantes dispersas por el código

Descripción: Los valores de plazos de préstamo y límites aparecían en múltiples sitios del código, lo que hacía que cualquier cambio de política requiriera buscar y modificar varios lugares a la vez.

Solución adoptada: Se creó la clase `ConfiguracionBiblioteca` como fuente única de todas las constantes del sistema.

Problema 5: Lógica de permisos dispersa

Descripción: Las comprobaciones de acceso tendían a aparecer dispersas por la interfaz y otras partes del código, dificultando su mantenimiento y aumentando el riesgo de inconsistencias.

Solución adoptada: Los métodos `es_admin()` y `es_bibliotecario_o_superior()` se implementaron directamente en la clase `Empleado`, centralizando toda la lógica de permisos en el modelo.

13. Conclusiones

El desarrollo del proyecto Reto Biblioteca POO ha permitido al equipo aplicar de forma práctica y cohesionada todos los conceptos fundamentales de la Programación Orientada a Objetos en un caso de uso real y complejo.

- La POO es el paradigma más adecuado cuando el problema del mundo real contiene entidades naturales con propiedades y comportamiento propio. En este caso, libros, usuarios y préstamos son objetos que encajan directamente en el modelo.
- Las decisiones de diseño tienen más impacto a largo plazo que el número de líneas de código. Una arquitectura bien pensada desde el inicio reduce drásticamente el esfuerzo de mantenimiento y evolución.
- La separación de responsabilidades es fundamental para crear software profesional y duradero. Cada capa y cada clase debe tener una única razón para cambiar.
- Las clases abstractas son una herramienta poderosa para prevenir errores futuros y garantizar la consistencia del sistema frente a nuevas incorporaciones.
- La validación debe residir en el modelo, no en la interfaz, para garantizar la integridad de los datos con independencia de la fuente que los origine.
- La configuración centralizada y el principio DRY evitan inconsistencias y facilitan enormemente el mantenimiento del sistema ante cambios de política o requisitos.
- El polimorfismo, aplicado correctamente, reduce el acoplamiento y permite que el sistema crezca añadiendo nuevas clases sin modificar el código existente.

Reflexión final: este proyecto nos ha enseñado que programar bien no consiste solo en hacer que el código funcione hoy, sino en construir algo que pueda evolucionar mañana con el mínimo esfuerzo. Las decisiones que tomamos al diseñar determinan la calidad del sistema durante toda su vida útil.